

Decorator



Notas organizadas a partir dos slides e das anotações de aula, Setembro/2026.

Disciplina: Métodos Avançados de Programação (MAP), UEPB — UEPB, 26.2

Professora: Luciana de Queiroz Leal Gomes

Referências: GAMMA, HELM, JOHNSON e VLISSIDES, *Padrões de Projeto* ·
FREEMAN e FREEMAN, *Use a Cabeça! Padrões de Projetos*

Material Auxiliar

Padrões de Projeto GoF

- [Decorator no Refactoring Guru](#)

1. O problema: um sistema corporativo de notificações

Imagine um sistema corporativo de notificações. O sistema envia e-mails, SMS e push notifications. Além disso, algumas notificações precisam de logs, criptografia, validação, monitoramento e retry automático.

Duas famílias de variação convivem nesse cenário: o **canal de envio** (e-mail, SMS, push) e as **funcionalidades extras** que podem ou não acompanhar cada envio (log, criptografia, validação, monitoramento, retry). **A escolha do canal é um problema já visto: pode ser resolvida com Strategy, trocando o algoritmo de envio conforme o canal.** O problema é o segundo eixo: como combinar, de forma livre, um número variável de funcionalidades extras em cima de um envio.

1.1 A solução tradicional: uma subclasse por combinação

A tentativa natural é criar uma hierarquia de `Notificacao` com uma subclasse para cada combinação: `Email`, `EmailComLog`, `EmailComCriptografia`, `EmailComLogECriptografia`, `Sms`, `SmsComLog`, e assim por diante.

Quando se usa herança para adicionar funcionalidades, cada combinação vira uma classe. Isso significa que a herança "congela" as combinações em tempo de compilação, resultando em comportamentos rígidos: uma vez definida a hierarquia, não é possível montar uma combinação nova sem escrever uma classe nova e recompilar o sistema.

O ponto central é: **a explosão de classes não é a causa do problema, ela é a consequência de tentar combinar comportamentos usando herança.** Aqui as funcionalidades extras não são alternativas entre si (como os tipos de frete), são combináveis, o que multiplica ainda mais o número de combinações possíveis.

Um cenário só com o e-mail já mostra a escala do problema: `EmailComLog`, `EmailComCriptografia`, `EmailComRetry`, `EmailComLogECriptografia`, `EmailComLogRetry`, `EmailComCriptografiaRetry`, `EmailComLogCriptografiaRetry`.

Com apenas três funcionalidades extras (log, criptografia, retry) e um único canal, já são sete combinações possíveis (todas as combinações não vazias de três itens); com três canais e cinco funcionalidades extras, o número de subclasses necessárias para cobrir todas as combinações cresce exponencialmente.

1.2 Repensando o problema

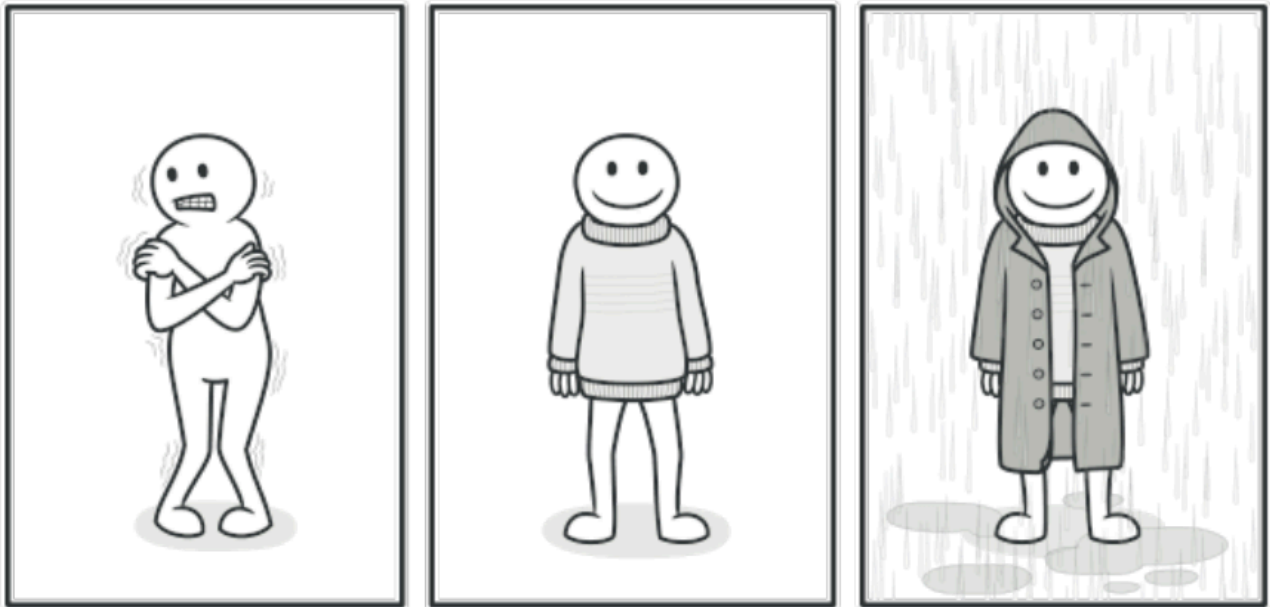
A pergunta que reorienta a aula: e se a solução não fosse criar mais classes? E se, em vez disso, log fosse uma funcionalidade independente, criptografia fosse uma funcionalidade independente, retry fosse uma funcionalidade independente, e essas funcionalidades pudessem ser combinadas em tempo de execução?

Se cada funcionalidade puder "envolver" o notificador original, é possível adicionar comportamentos sem criar uma subclasse para cada combinação. Como consequência dessa mudança de perspectiva, passa a ser possível adicionar, mover ou combinar comportamentos em tempo de execução: adicionar funcionalidades dinamicamente, combinar comportamentos livremente e trocar funcionalidades em runtime. Isso segue o **princípio Open/Closed**: uma classe deve ficar aberta para extensão e fechada para modificação, ou seja, ganha-se comportamento novo criando código novo, sem editar o que já existe e já funciona.

2. O padrão Decorator

O padrão Decorator surge exatamente para resolver combinações de funcionalidades de forma flexível e elegante. A proposta central do padrão, na formulação usada em aula: **coloque cada responsabilidade em um objeto separado**. Ou seja: cada funcionalidade vira um objeto, essas funcionalidades podem ser combinadas dinamicamente, é possível adicionar ou remover comportamentos em tempo de execução, evita-se a explosão de subclasses, e usa-se composição em vez de herança na classe principal.

A definição oficial do GoF: o padrão Decorator dinamicamente agrega responsabilidades adicionais a um objeto, fornecendo uma alternativa flexível ao uso de subclasses para extensão de responsabilidades (GAMMA et al., 2000). O nome alternativo do padrão no catálogo original é **Wrapper** (envoltório), porque cada decorador literalmente embrulha o objeto que decora.



A motivação original do GoF ilustra a mesma ideia num domínio diferente: um framework de interface gráfica em que se quer acrescentar bordas ou barra de rolagem a um componente visual sem herdar uma classe para cada combinação de borda e rolagem, e sem que o cliente perca a capacidade de decidir, em tempo de execução, quais dessas decorações aplicar a cada instância (GAMMA et al., 2000).

3. Características e participantes

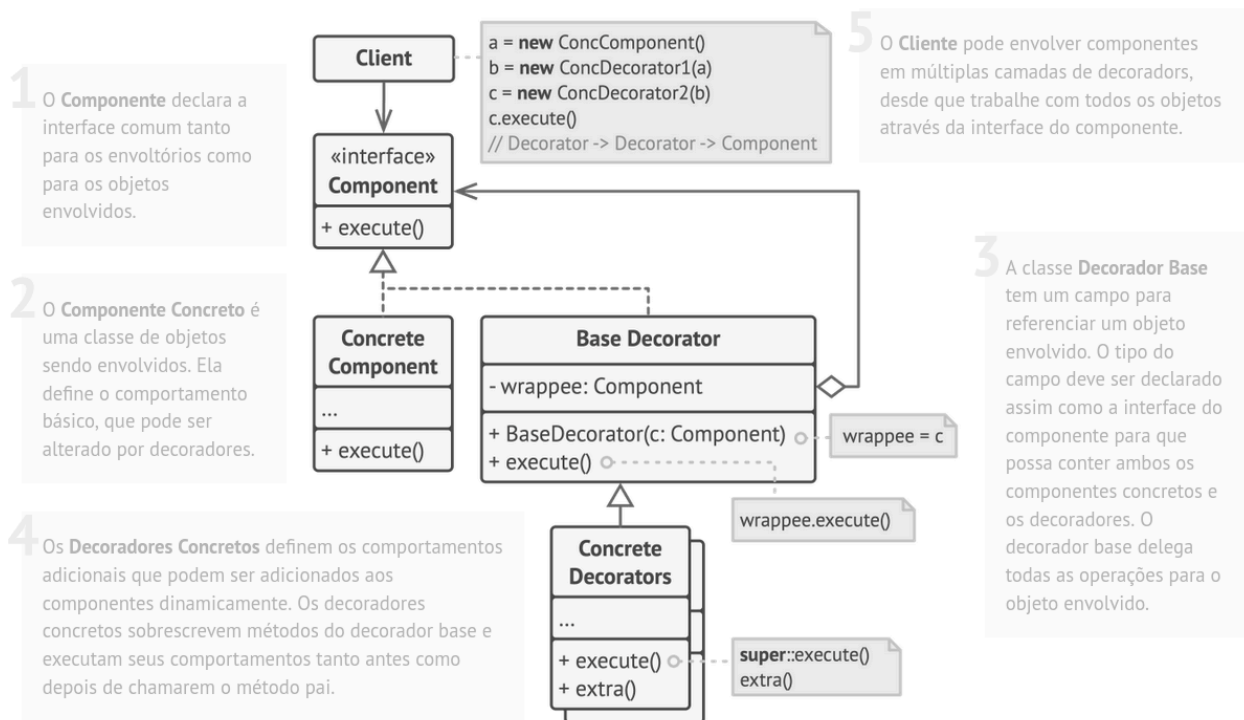
Características do padrão: usa composição, evita explosão de subclasses, mantém o código flexível, e permite extensão sem alterar código existente.

Os quatro participantes:

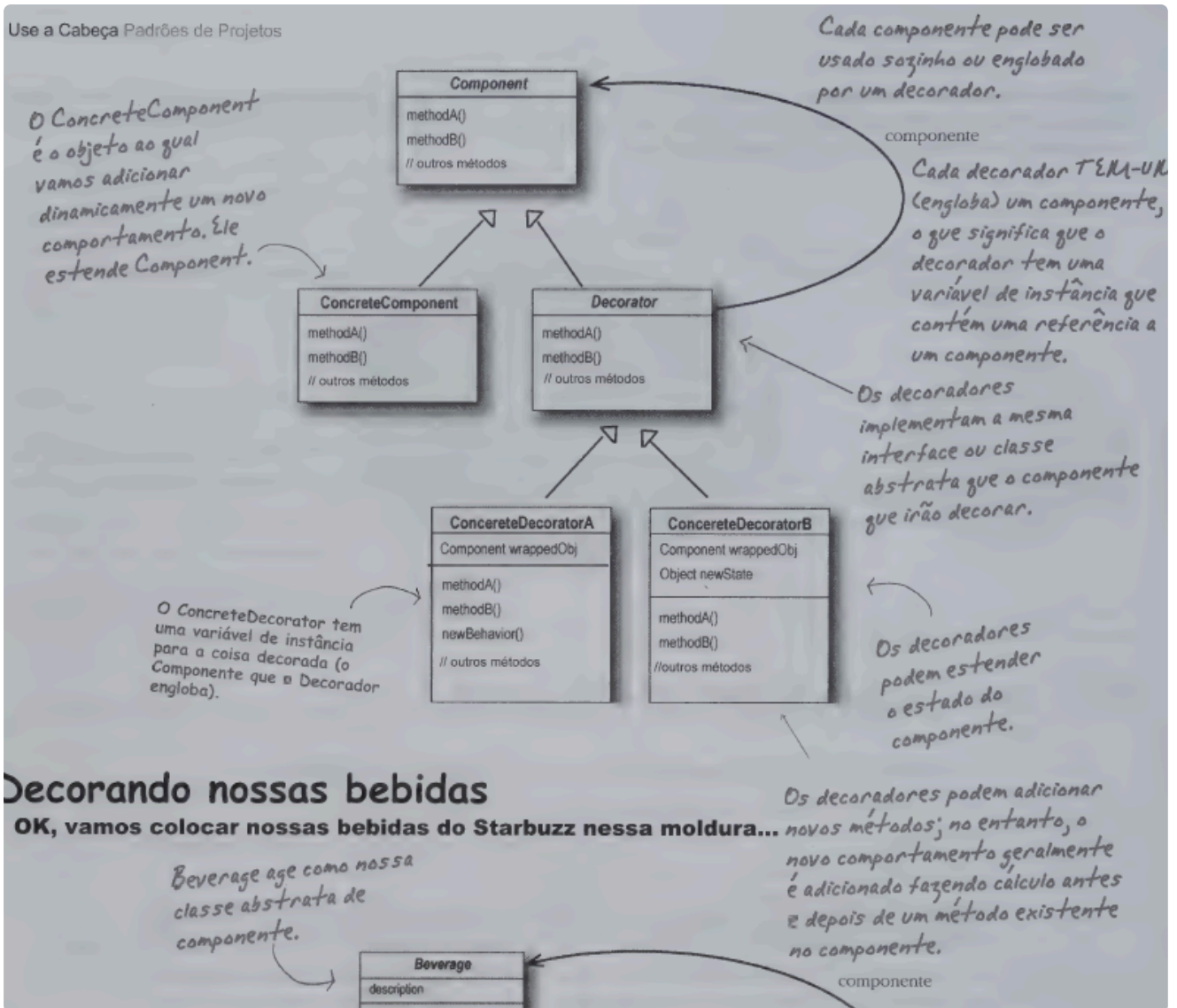
- **Component**
- **ConcreteComponent**
- **Decorator**
- **ConcreteDecorator**

O decorador implementa a mesma interface do objeto original: essa é a peça-chave que faz tudo funcionar, porque permite que um cliente trate um objeto decorado exatamente como

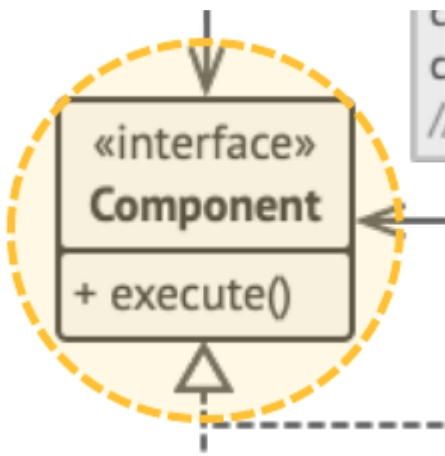
trataria o objeto original, sem precisar saber se está lidando com um ou com o outro.



O comentário ao lado do **Client**, no diagrama, mostra como as camadas se empilham na criação do objeto: `a = new ConcComponent(); b = new ConcDecorator1(a); c = new ConcDecorator2(b); c.execute()`, com a observação de que a cadeia de chamadas percorre **Decorator** → **Decorator** → **Component**.



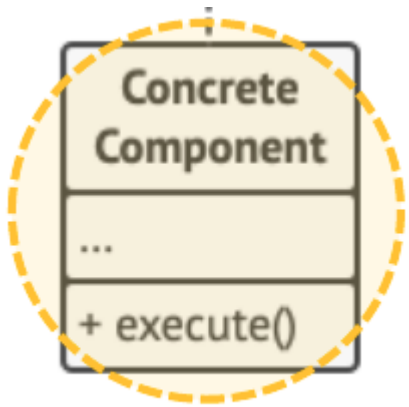
3.1 Component



É a interface comum do sistema. Define as operações que podem ser decoradas.

Responsabilidade: definir o contrato comum, permitir polimorfismo, e garantir que decoradores e objetos reais sejam usados da mesma forma.

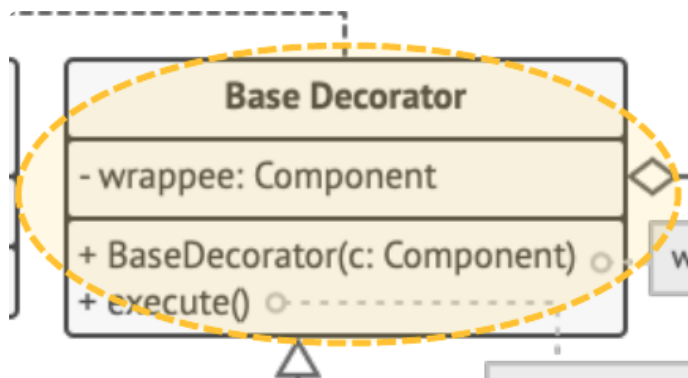
3.2 ConcreteComponent



É o objeto principal, o objeto base. Representa o comportamento original, antes de qualquer decorador ser aplicado.

Responsabilidade: implementar a funcionalidade real e servir de núcleo para os decoradores.

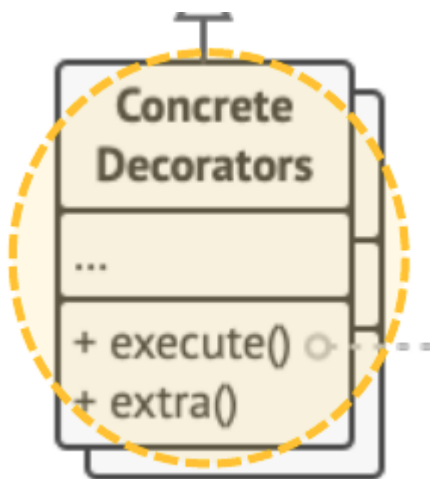
3.3 Decorator (BaseDecorator)



É a classe base dos decoradores. Ela também implementa a interface Component, e é justamente essa implementação compartilhada que garante que **um decorador possa substituir, do ponto de vista de quem o usa, tanto o componente concreto quanto outro decorador.**

Responsabilidade: **manter uma referência para outro Component (o objeto que está sendo decorado, às vezes chamado de *wrappee*), delegar chamadas a esse objeto, e servir de base para os decoradores concretos.**

3.4 ConcreteDecorator

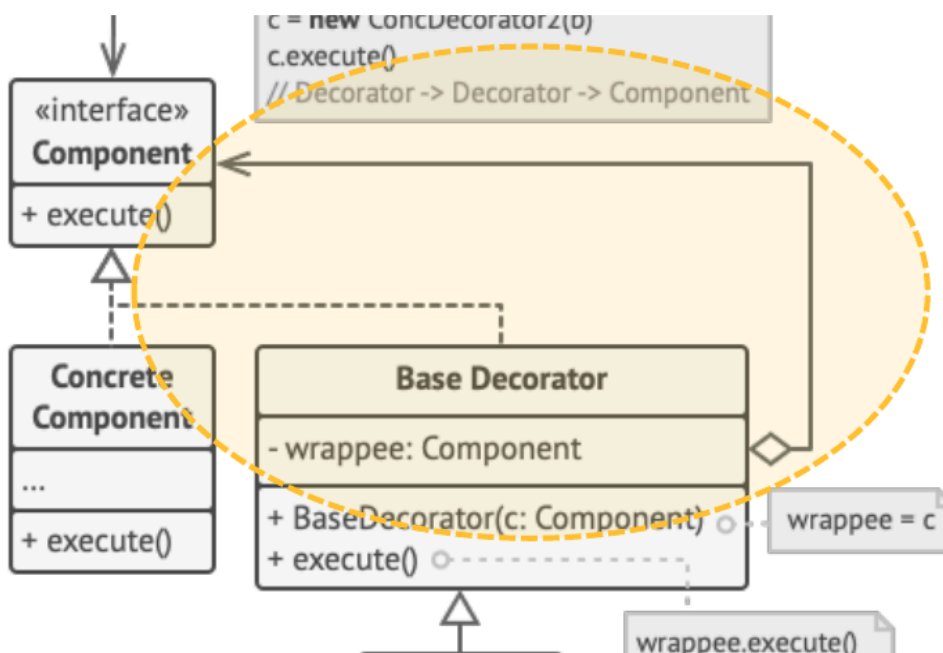


São os comportamentos adicionais propriamente ditos. Cada decorador concreto adiciona uma responsabilidade nova.

Responsabilidade: adicionar funcionalidades, executar comportamento antes e/ou depois de delegar a chamada ao objeto decorado, e manter a interface compatível com a de Component.

Esse é o pulo do 🐱 que faz com que o decorator funcione: quando eu chamo a minha função da interface (`.execute()` no exemplo genérico) dentro dela eu vou chamar o `.execute()` do meu atributo (que também é um decorator e tem seu próprio `execute` que chama o seu atributo)!! Isso até eu chegar do concrete decoratos que não um Decorator como atributo

3.5 Composição recursiva

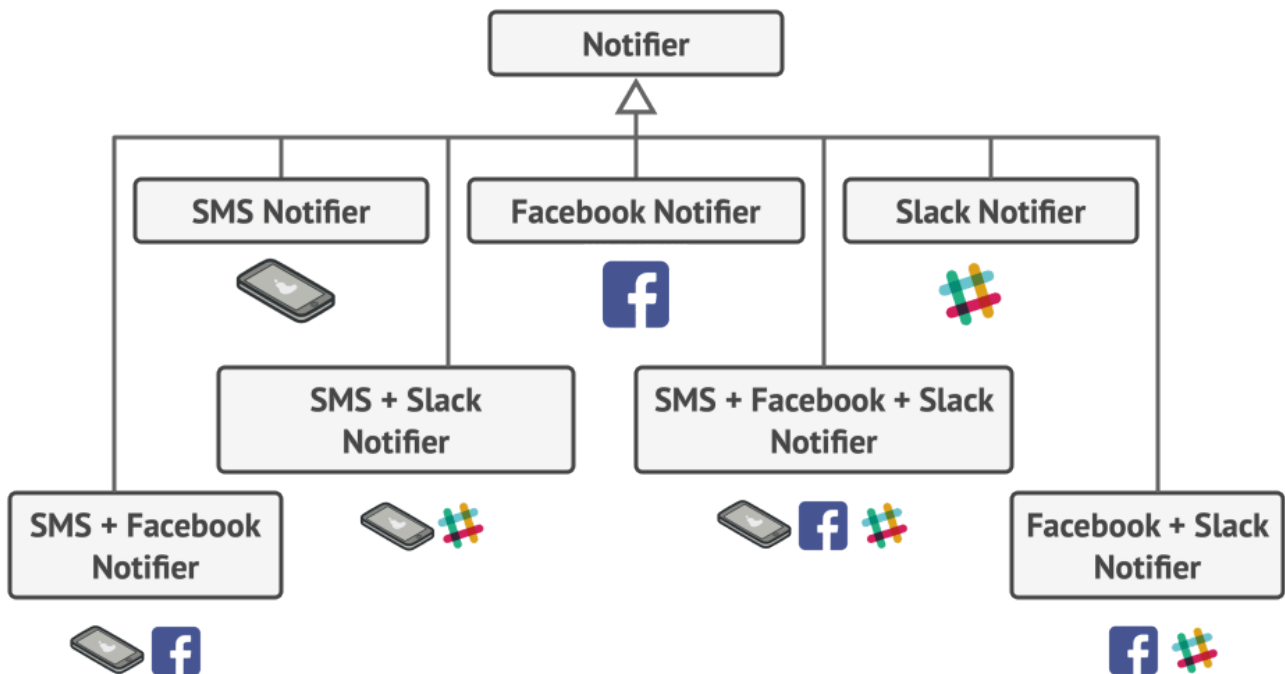


O ponto que fecha a estrutura: um objeto possui, internamente, uma referência para outro objeto do mesmo tipo lógico ou da mesma abstração. Um objeto pode conter outro objeto que também pode conter outro objeto, formando uma estrutura encadeada, análoga a uma linked list. A estrutura pode crescer indefinidamente porque o objeto referencia a própria abstração (o Decorator guarda uma referência a Component, e o próprio Decorator também é um Component), e é exatamente essa auto-referência que permite adicionar

quantos comportamentos forem necessários, em qualquer ordem, sem alterar nenhuma classe existente.

4. Exemplo trabalhado: notificações por e-mail

4.1 Sem decorator, para contraste



```
public abstract class Notificador {
    void enviar(String mensagem);
}

public class EmailNotificador extends Notificador {
    @Override
    public void enviar(String mensagem) {
        System.out.println("Enviando email");
    }
}
```

A partir daqui, cada combinação de funcionalidade exige uma subclasse nova:

```
public class EmailComCriptografia extends Notificador {
    @Override
    public void enviar(String mensagem) {
        mensagem = "###" + mensagem + "###";
        System.out.println("Enviando email");
    }
}
```

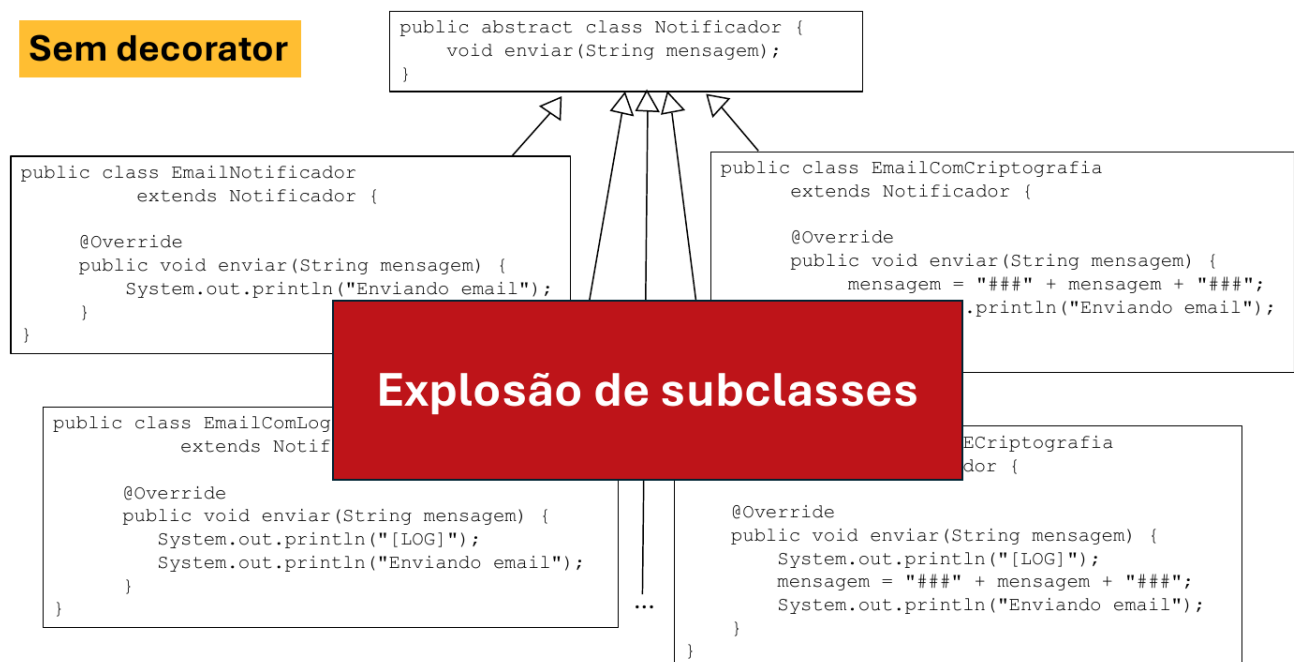
```

public class EmailComLog extends Notificador {
    @Override
    public void enviar(String mensagem) {
        System.out.println("[LOG]");
        System.out.println("Enviando email");
    }
}

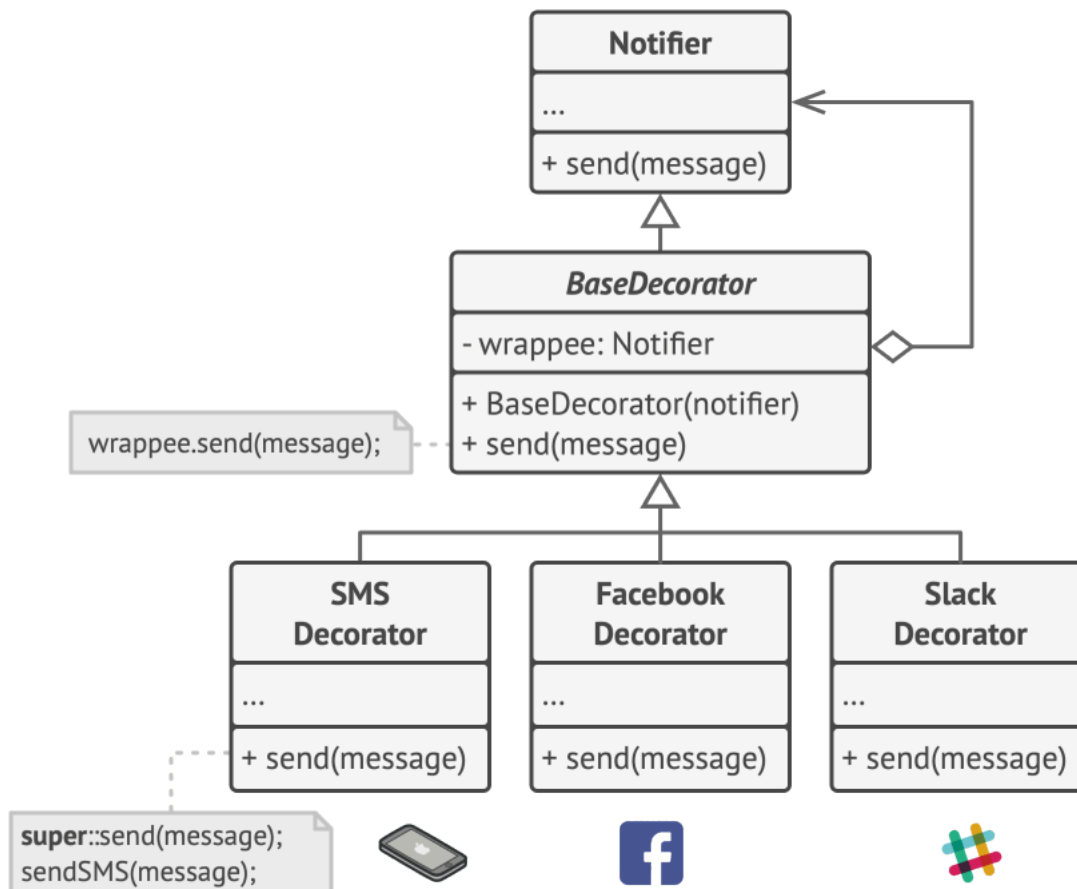
public class EmailComLogECriptografia extends Notificador {
    @Override
    public void enviar(String mensagem) {
        System.out.println("[LOG]");
        mensagem = "###" + mensagem + "###";
        System.out.println("Enviando email");
    }
}

```

Já com quatro classes só para uma funcionalidade de e-mail (sem contar SMS e push), fica visível a explosão de subclasses anunciada na seção 1: cada nova combinação de log, criptografia e retry exigiria copiar e recombinar a lógica manualmente em uma classe nova.



4.2 Reescrevendo com Decorator



```

// Component
public interface Notificador {
    void enviar(String mensagem);
}

// ConcreteComponent
public class EmailNotificador implements Notificador {
    @Override
    public void enviar(String mensagem) {
        System.out.println("Enviando email");
    }
}

// Decorator
public abstract class NotificadorDecorator implements Notificador {
    protected Notificador notificador; // protected porque essa classe
    abstrata será herdada

    public NotificadorDecorator(Notificador notificador) {
        this.notificador = notificador;
    }
}

// ConcreteDecorator. Não preciso do implements porque a base já
implementa
public class LogDecorator extends NotificadorDecorator {

```

```

    public LogDecorator(Notificador notificador) {
        super(notificador);
    }

    @Override
    public void enviar(String mensagem) {
        System.out.println("[LOG] Iniciando envio");
        notificador.enviar(mensagem);
    }
}

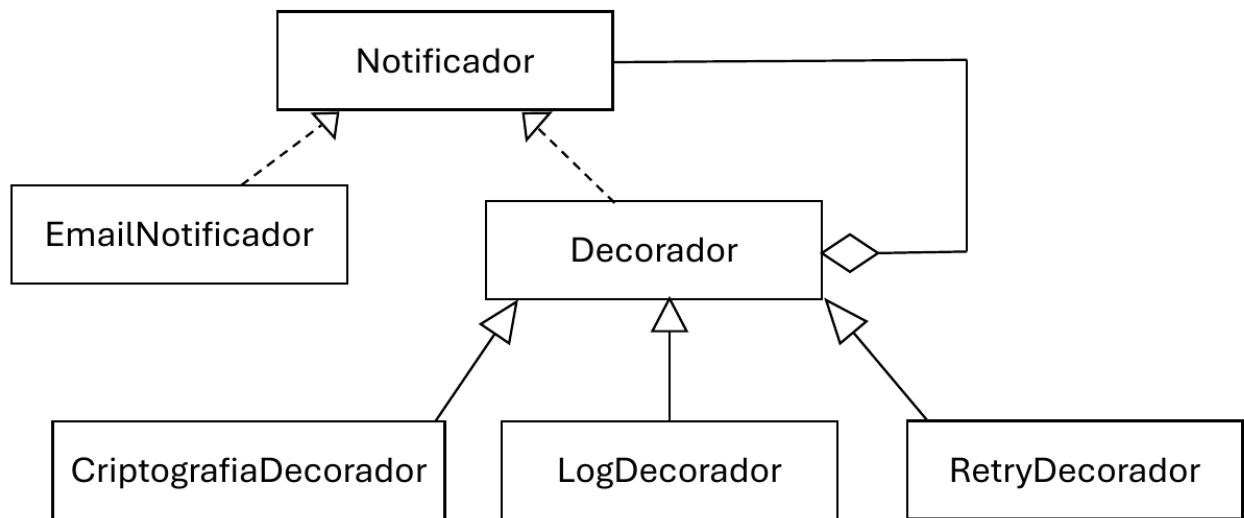
// ConcreteDecorator
public class CriptografiaDecorator extends NotificadorDecorator {
    public CriptografiaDecorator(Notificador notificador) {
        super(notificador);
    }

    @Override
    public void enviar(String mensagem) {
        mensagem = "###" + mensagem + "###";
        notificador.enviar(mensagem);
    }
}

```

NotificadorDecorator guarda a referência ao Notificador que está sendo decorado (o *wrappee* do diagrama genérico) e implementa Notificador; é essa dupla condição, ser um Notificador e ao mesmo tempo guardar um Notificador dentro de si, que permite empilhar decoradores livremente. LogDecorator e CriptografiaDecorator não sabem se notificador é o EmailNotificador original ou outro decorador: para eles, é sempre apenas "um Notificador", e cada um só se preocupa em fazer a sua parte antes de repassar a chamada adiante.

Comparando com a hierarquia da seção 4.1: em vez de quatro classes fixas para cobrir quatro combinações, **agora são três classes pequenas** (LogDecorator , CriptografiaDecorator e, se necessário, RetryDecorator) que, combinadas, cobrem qualquer uma das sete combinações possíveis vistas na seção 1.1, e continuam cobrindo qualquer combinação nova (por exemplo, log duas vezes) sem que nenhuma classe precise ser criada ou alterada.

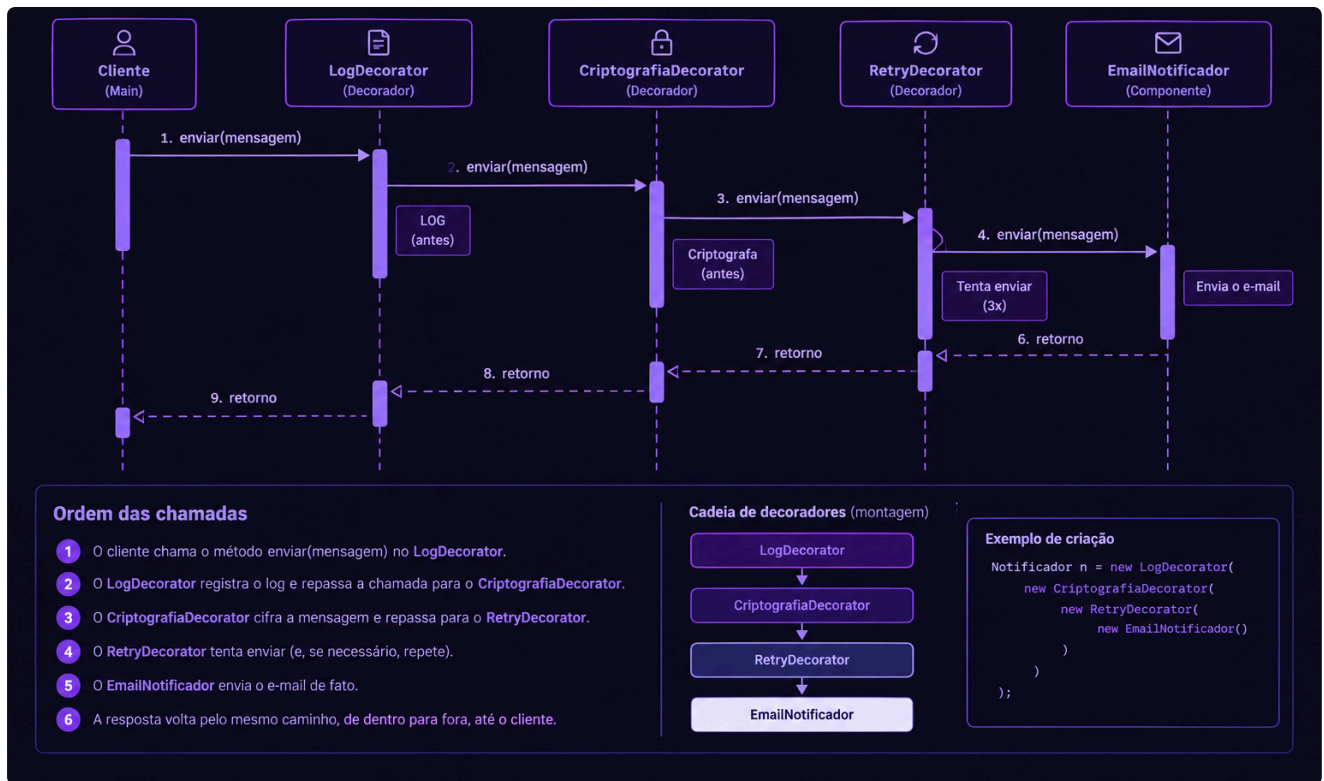


4.3 Montando e usando a cadeia de decoradores

```
public class Main {
    public static void main(String[] args) {
        Notificador n = new LogDecorador(
            new CriptografiaDecorador(
                new EmailNotificador()
            )
        );

        n.enviar("Pagamento aprovado");
    }
}
```

A ordem de execução, de fora para dentro na criação e de dentro para fora na entrega final, mas sempre respeitando a ordem de aninhamento: primeiro o LOG executa (é o decorador mais externo, então é o primeiro a receber a chamada `enviar`), depois a mensagem é criptografada (o decorador seguinte na cadeia), e só então o e-mail é efetivamente enviado (o `ConcreteComponent`, no centro da cadeia).



Fica, então, um encadeamento equivalente a um `new` dentro de outro `new`: cada decorador só existe amarrado a um `Notificador` interno, e a chamada só produz efeito quando `enviar` é de fato invocado na ponta (`n.enviar(...)`); e a ordem em que os decoradores são aplicados importa, porque decide a ordem em que os efeitos colaterais de cada um acontecem. Trocar a ordem de `LogDecorator` e `CriptografiaDecorator` na montagem mudaria qual acontece primeiro, o registro em log ou a cifragem da mensagem, ainda que o resultado final (mensagem cifrada, log registrado, e-mail enviado) pareça semelhante.

Esse diagrama mostra o sistema já com um terceiro decorador (`RetryDecorator`) acrescentado à cadeia, sem que nada nas classes já existentes (`Notificador`, `EmailNotificador`, `Decorator`, `CriptografiaDecorator`, `LogDecorator`) precise mudar: a nova classe só implementa `Decorator` e passa a poder ser combinada livremente com as demais.

5. Herança e composição, revisitadas

Característica	Herança	Decorator
Estrutura	Fixa	Dinâmica
Número de classes	Muitas subclasses	Poucas classes
Flexibilidade	Menos flexível	Mais flexível

Característica	Herança	Decorator
Acoplamento	Forte	Baixo
Quando a combinação é decidida	Em compilação	Em execução

O Decorator entrega uma estrutura dinâmica, com poucas classes e mais flexibilidade, exatamente como o próprio Strategy fez para algoritmos intercambiáveis e o Observer fez para notificação de mudanças de estado. Os três padrões resolvem o mesmo problema de fundo (dependência excessiva de classes concretas, decidida cedo demais) com a mesma ferramenta de fundo (composição sobre uma interface estável), cada um aplicado a uma forma diferente de variação: algoritmo único escolhido (Strategy), muitos dependentes reagindo a um evento (Observer), e camadas de comportamento empilhadas em torno de um núcleo (Decorator).

O GoF chama essa vantagem de flexibilidade maior que a herança estática: como as responsabilidades podem ser associadas e dissociadas de um objeto em tempo de execução, basta compor e decompor decoradores em vez de multiplicar classes; a herança, para cobrir o mesmo leque de combinações, exigiria uma nova classe para cada combinação adicional. O catálogo original ainda destaca uma vantagem sutil: como cada decorador é uma instância independente, é simples aplicar a mesma decoração mais de uma vez (por exemplo, dois `LogDecorator` em sequência), algo que herdar de uma classe duas vezes não permite fazer de forma direta (GAMMA et al., 2000).

O livro *Use a Cabeça!* resume o mesmo contraste como um princípio de projeto que já vinha sendo construído desde o Strategy e o Observer: as classes devem ficar abertas para extensão, mas fechadas para modificação, e o Decorator é uma das formas mais diretas de obter essa propriedade, porque estender o comportamento de um objeto passa a significar escrever uma classe decoradora nova, nunca tocar nas classes já testadas e em produção (FREEMAN; FREEMAN, 2009).

6. Vantagens e desvantagens

- **Vantagens:** adiciona funcionalidades dinamicamente; evita explosão de subclasses; favorece composição em vez de herança; mantém responsabilidades independentes umas das outras; permite combinação flexível de funcionalidades.
- **Desvantagens:** aumenta a quantidade de objetos no sistema (cada camada de decoração é um objeto a mais em tempo de execução); a ordem dos decoradores pode impactar o comportamento final e a estrutura fica mais complexa de acompanhar para quem está começando, justamente porque o comportamento de uma chamada só se

revela percorrendo toda a cadeia de objetos encadeados, em vez de estar concentrado em uma única classe.

O GoF acrescenta duas consequências:

1. Um objeto decorado não é idêntico, do ponto de vista de identidade, ao componente original que ele envolve, então não se deve depender de comparação de identidade (nem, em Java, de `==`) ao trabalhar com objetos decorados;
2. Sistemas que usam Decorator tendem a ficar compostos por muitos objetos pequenos e parecidos entre si, que diferem apenas na forma como estão interconectados e não nas suas classes, o que é conveniente para quem já entende a estrutura, mas pode ser difícil de depurar para quem não a conhece bem (GAMMA et al., 2000). É essencialmente a mesma ideia da "estrutura mais complexa para iniciantes" dos slides da professora, com uma explicação mais precisa do motivo: a dificuldade não está na lógica de cada decorador isoladamente (que costuma ser simples), está em acompanhar a cadeia inteira.

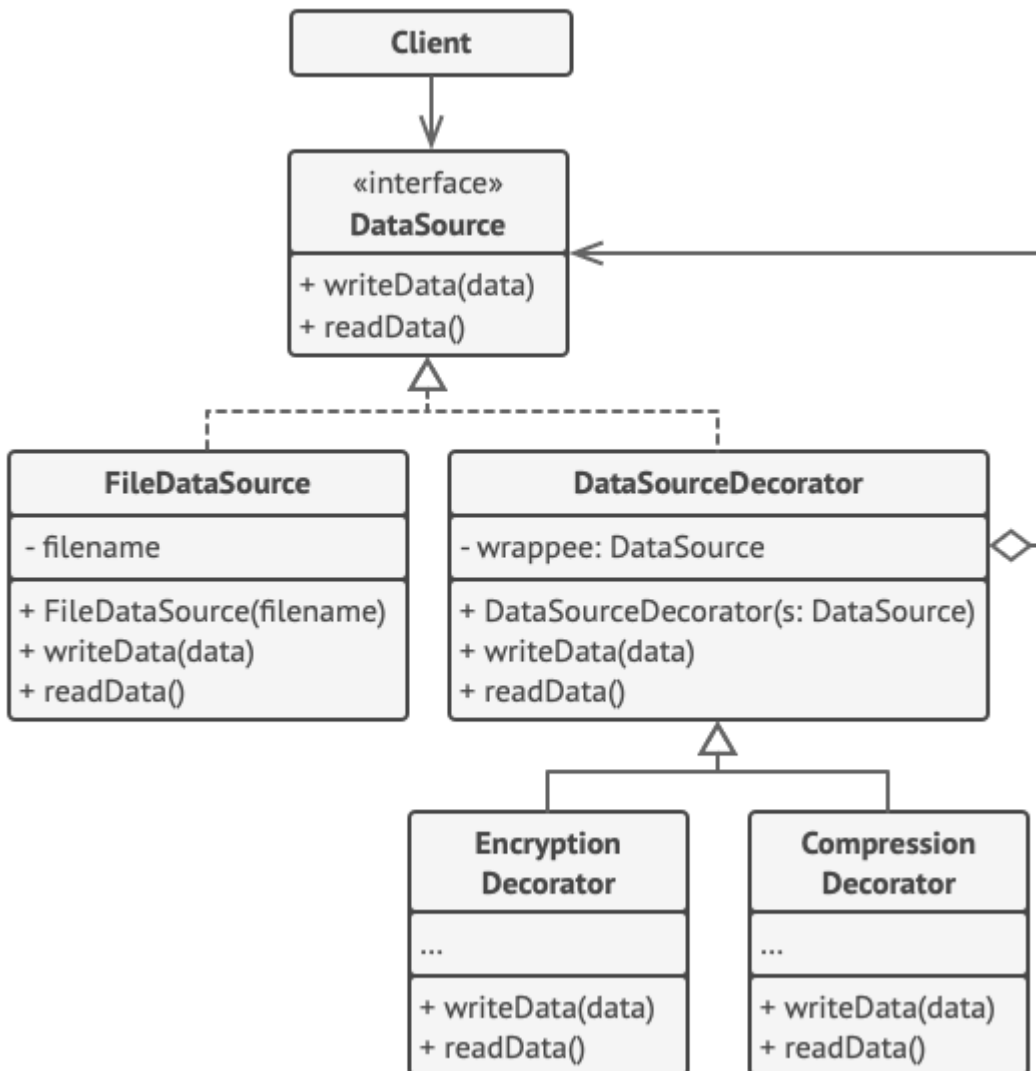
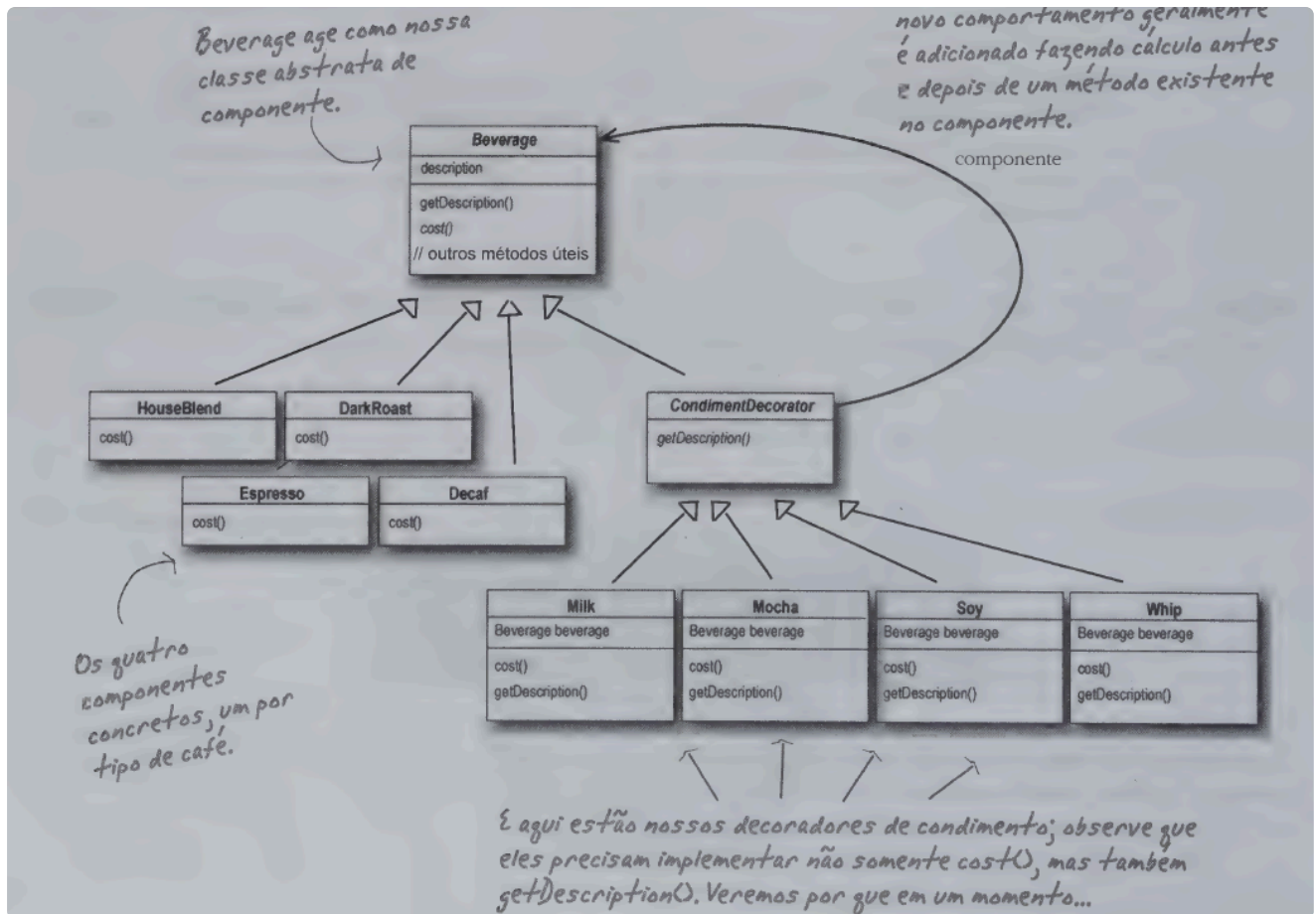
7. Quando usar e quando não usar

🐱 Use Decorator quando for preciso combinar funcionalidades livremente, e quando diferentes combinações dessas funcionalidades forem necessárias ao longo do tempo (o cenário de log, criptografia e retry combináveis à vontade é o caso típico).

🐱 Não use Decorator quando o comportamento não varia, ou quando o sistema precisa ser muito simples: nesses casos, o decorador só adiciona mais objetos, mais indireção e mais complexidade, sem trazer nenhum benefício real. Se só existe uma única forma de enviar notificação, sem log, sem criptografia e sem retry, nem vale a pena montar a estrutura de decoradores: o `if/else` mais simples, ou mesmo só o `EmailNotificador` sozinho, resolve.

Assim como no Strategy, a decisão de quais decoradores aplicar, e em que ordem, é normalmente resolvida em tempo de execução, deixando para o código cliente (o `main`, ou uma camada de configuração) a responsabilidade de montar a cadeia que faz sentido para cada situação.

8. Mais exemplos



Class Diagram of Decorator Design Pattern

